Middle East Technical University | Department of Computer Engineering

# CENG 499

## Introduction to Machine Learning

Fall 2026-2027

## Homework 1

Due date: 03 November 2026, Tuesday, 23:55

# Objectives

This assignment aims to fulfill the following objectives:

- To familiarize you with the backpropagation algorithm, which is crucial for neural network learning, and enable you to gain hands-on experience in solving problems with this method by employing multilayer perceptrons (feed-forward networks).
- To familiarize you with the basic operations and functionalities of PyTorch along with its high-level (abstract) features.
- To familiarize you with the hyperparameter search (grid search) for model selection.
- To give you an idea of how to report statistically significant results when stochasticity (randomness) is involved while applying an ML method to a particular task.

# Part 0

The learning (statistical learning) problem can be considered as "finding a desired dependence using a limited number of observations" (Vapnik Chapter 1). The solution to this problem can be formulated as a function identification/search problem of the following form (Vapnik Chapter 1):

$$\alpha^* = \arg\min_{\alpha} R(\alpha) = \int L(y, f(x, \alpha)) p(x, y)$$

Where $f(x, \alpha)$ is the function implemented by a method (learning machine, statistical model), $\alpha$ is the set of particular parameter/hyperparameter values (e.g., for an multilayer perceptron, $\alpha$ is consists of the network parameters (weights and biases in layers) along with hyperparameters such as number of layers, number of nodes in layer, activation functions used in layers, the learning rate utilized during network training, .etc), $y$ is the ground truth information for a particular data instance $x$, $L$ is the loss function measuring the discrepancy between ground truth and the output of the method, $p(x, y)$ is the joint probability function for a data instance $x$ and its associated information $y$ (e.g., label), $R$ is the expected loss (risk) associated with a particular hyperparameter/parameter value $\alpha$. The mathematical formulation states that to solve the learning problem, the parameter/hyperparameter values of the method (learning machine) that attain the minimum expected risk value need to be found. Since, in practice, we have only

a limited number of samples (e.g., data points), the expected risk formulation can be approximated via the empirical risk (Vapnik Chapter 1):

$$\alpha^* = \arg\min_{\alpha} R_{emp} = \frac{1}{N}\sum_{i=1}^{N} L(y_i, f(x_i, \alpha))$$

. Typically, there are three main types of learning problems: pattern recognition (classification), regression, and density estimation (Vapnik Chapter 1). Within the scope of this assignment, our focus will be on classification and regression problems.

The learning problem is ill-posed (Alpaydın Chapter 2) due to the fact that there is not a single unique solution for it. This is the basic reason why there are many machine learning models/methods/algorithms that differ in the assumptions (hypotheses) they make regarding this problem. Hyperparameters of models/methods/algorithms determine the space of functions/hypotheses implemented by the models/methods, whereas parameters determine the function/hypothesis implemented. For instance, the number of layers in a multilayer perceptron (MLP) affects the possible set of functions that can be represented/learned (the more layers, the more complex functions can be represented). Or, while applying gradient descent with the MLP, the learning rate and iteration number (epoch number) limit what parameter values can be reached during learning; hence, they indirectly impose a set of functions that can be learned. Within the scope of machine learning, the main goal of learning is to obtain models that can generalize to unseen cases (e.g., determining labels of unseen data points) by using all currently available data. To obtain the best parameter/hyperparameter values for methods (estimating model parameters/hyperparameters ($\alpha$), or model tuning) to solve a learning problem, there are mainly two estimation approaches: frequentist estimation and Bayesian estimation. A frequentist estimation obtains a single set of parameter/hyperparameter values, whereas Bayesian estimation aims to define a probability distribution over parameter/hyperparameter values to assess the uncertainty in parameter values for solving a given problem (Bishop Chapter 1). One of the most utilized frequentist estimation approaches is the maximum likelihood estimation (MLE) (Alpaydın Chapter 4, Bishop Chapter 1). MLE aims to find $\theta^*$ such that $p(D|\theta)$ (the joint probability of data instances for a given parameter $\theta$) is maximized for a given set of data instances, whereas the Bayesian estimation approach models $P(\theta|D)$ (the probability of the parameter for a given dataset $D$) and uses this probability distribution function for solving learning problems. The Bayesian approach includes MLE as a special case:

$$p(\theta|D) = \frac{p(D|\theta)p(\theta)}{p(D)}$$

As can be seen, the Bayesian estimation approach treats model parameters/hyperparameters as random variables and aims to estimate a probability distribution over them by utilizing a given dataset. Whereas MLE finds a point estimate (single value, no probability distribution over parameter values) for model parameters/hyperparameters. Both approaches have their own merits, advantages, and disadvantages. However, due to the Bayesian approach's computational demands, MLE is generally utilized in solving machine learning problems.

### 0.0.1 Maximum Likelihood Estimation

Suppose that we have a set of data points denoted $D = \{\vec{x_i} | i = 1..N\}$ where $i^{th}$ data instance is represent with $\vec{x_i}$ (as a high dimensional vector). Let's assume that each data point $\vec{x_i}$ is generated by an unknown data generation process whose generation probability distribution is $q(\vec{x_i})$ (we assume data instances are identically distributed via $q$). To approximate this unknown data probability distribution, we can build a mathematical model that models $p(\vec{x_i}|\theta)$, where $\theta$ is the model parameter. Once we obtain/learn $\theta$, we can use it to measure the likelihood of new data instances.

To approximate $q(\vec{x_i})$ with $p(\vec{x_i}|\theta)$, MLE finds a parameter value such that the likelihood of data

points is maximized. Mathematically:

$$\theta_{MLE} = \arg\max_{\theta} p(D|\theta) \tag{1}$$

$$= \arg\max_{\theta} p(\vec{x_1}, \vec{x_2}, ...\vec{x_N}|\theta) \tag{2}$$

Further, if we assume the data points are independently distributed:

$$\theta_{MLE} = \arg\max_{\theta} p(\vec{x_1}|\theta)p(\vec{x_2}|\theta)...p(\vec{x_N}|\theta) \tag{3}$$

$$= \arg\max_{\theta} \prod_{i=1}^{N} p(\vec{x_i}|\theta) \tag{4}$$

When data instances have associated labels, the data set can be represented as $\bar{D} = \{(\vec{x_i}, label_i)|i = 1..N\}$ where $\vec{x_i}$ is the i$^{\text{th}}$ data instance and $label_i$ is the label associated with it. So in this case, we can assume data instances and their labels are generated with an unknown data generation process whose generation probability distribution is $q(\vec{x_i}, label_i)$. Similarly, we can build a model that represents $p(\vec{x_i}, label_i|\theta)$ and learn a parameter value that approximates $q(\vec{x_i}, label_i)$ via MLE. Mathematically:

$$\theta_{MLE} = \arg\max_{\theta} p(\bar{D}|\theta) \tag{5}$$

$$= \arg\max_{\theta} p((\vec{x_1}, label_1), (\vec{x_2}, label_2), ...(\vec{x_N}, label_N)|\theta) \tag{6}$$

Further, if we assume each data instance-label pair is independently distributed:

$$\theta_{MLE} = \arg\max_{\theta} p(\vec{x_1}, label_1|\theta), p(\vec{x_2}, label_2|\theta), ...p(\vec{x_N}, label_N|\theta) \tag{7}$$

$$= \arg\max_{\theta} \prod_{i=1}^{N} p(\vec{x_i}, label_i|\theta) \tag{8}$$

After learning $p(\vec{x_i}, label_i|\theta)$, we can label a new data instance $\vec{x_y}$. Suppose that a label value can take two values, $a$ and $b$, without loss of generality. To label $\vec{x_y}$, we can check the probability values: $p(\vec{x_y}, a|\theta)$ and $p(\vec{x_y}, b|\theta)$, since $p(\vec{x_i}, label_i|\theta)$ provides a probability value as to how likely a data instance and a label value coexist (co-generated). Thus, if $p(\vec{x_y}, a|\theta) > p(\vec{x_y}, b|\theta)$ ($a$ is more likely to be associated with $\vec{x_y}$), we can label $\vec{x_y}$ as $a$; otherwise, $b$ (if $p(\vec{x_y}, a|\theta) = p(\vec{x_y}, b|\theta)$, we can label it with either label or defer labeling). Furthermore, we can observe that the labeling can be done via the conditional probabilities $(p(label_i|\vec{x_y}), \theta)$ since:

$$p(\vec{x_y}, a|\theta) = \frac{p(a|\vec{x_y}, \theta)}{p(\vec{x_y}|\theta)}, p(\vec{x_y}, b|\theta) = \frac{p(b|\vec{x_y}, \theta)}{p(\vec{x_y}|\theta)}$$

While comparing, the common term $p(\vec{x_y}|\theta)$ can be eliminated. As a consequence, for labeling, we don't need to approximate the joint distribution $p(\vec{x_i}, label_i|\theta)$. Instead, we can try modeling $p(label_i|\vec{x_i}, \theta)$, which can reduce computation because $p(\vec{x_i}, label_i|\theta)$ includes more information and hence poses a harder learning problem compared to $p(label_i|\vec{x_i}, \theta)$. So, instead of learning about the coexistence of a data instance and a label via $p(\vec{x_i}, label_i|\theta)$, we can learn how an instance can be labeled via $p(label_i|\vec{x_i}, \theta)$. If we assume the unknown labeling procedure has a labeling distribution $q(label_i|\vec{x_i})$, we can build a model that represents $p(label_i|\vec{x_i}, \theta)$ and learn the parameter via MLE to approximate $q(label_i|\vec{x_i})$ (the unknown labeling probability distribution). We can separate the dataset $\bar{D}$ into two sets: $L = \{label_i|i = 1.N\}$ (labels) and $X = \{\vec{x_i}|i = 1..N\}$ (data instances). With the conditional function $p(label_i|\vec{x_i}, \theta)$, we aim to model:

$$\theta_{MLE} = \arg\max_{\theta} p(L|X, \theta) \tag{9}$$

$$= \arg\max_{\theta} p(label_1, label_2....label_N|\vec{x_1}, \vec{x_2}, ...\vec{x_N}, \theta) \tag{10}$$

Instead of maximizing the probability of observing individual distances as previously considered, here, we try to maximize the likelihood of observing the labels for given data instances. If we assume labels are independent of each other and the label is determined solely by the data instance it is associated with:

$$\theta_{MLE} = \arg\max_{\theta} p(label_1|\vec{x_1}, \theta)p(label_2|\vec{x_2}, \theta)...p(label_N|\vec{x_N}, \theta) \tag{11}$$

$$= \arg\max_{\theta} \prod_{i=1}^{N} p(label_i|\vec{x_i}, \theta) \tag{12}$$

Modeling/approximating/learning the likelihood of data instances ($p(\vec{x})$) is coined as the density estimation problem (Alpaydın Chapter 4), whereas modeling/approximating the unknown labeling procedure ($p(label|\vec{x})$) that leads to the emergence of label-data instance pairs (dataset) is coined as either a classification or regression problem depending on the values that labels can take (Vapnik Chapter 1). MLE can be used to solve all these three learning problems.

Since our focus is on classification and regression problems for this assignment, the following subsections discuss the solution strategies for these problems.

## 0.1 Solution Strategies for a Classification Problem (Bishop Chapter 1)

1. Learn/estimate the joint probability function $p(\vec{x}, label)$ via a given dataset and use this estimated probability function to get conditional class label probabilities:

$$p(label|\vec{x}) = \frac{p(\vec{x}, label)}{p(\vec{x})} = \frac{p(\vec{x}, label)}{\sum_{i=1}^{K} p(\vec{x}, label = l_i)}$$

Where $l_i$ denotes class label values of the given classification problem with $K$ labels. After calculating this conditional probability for every class label ($p(label = l_1|\vec{x}), p(label = l_2|\vec{x}), ...p(label = l_K|\vec{x})$), we can assign the label $l_i$ that attains the highest probability score to a given data instance $\vec{x}$ to minimize the error probability as discussed previously. As a broader decision strategy, in a given problem, if we are given risk/loss scores for each labeling decision (e.g., labeling an instance as $l_1$), we can utilize the probability score associated with each label to calculate the expected risk score for our labeling decisions. Let $R(\ alpha_i|\ Vecx)$ be the expected risk/loss score when we label a data instance $\vec{x}$ as $l_i$ (Alpaydın Chapter 3):

$$R(\alpha_i|\vec{x}) = \sum_{j=1}^{K} \lambda_{ij} * p(label = l_j|\vec{x})$$

where $\lambda_{ij}$ is the loss/risk incurred if we label $\vec{x}$ as $l_i$ when it actually belongs to the class $l_j$. For instance, in a medical application, labeling a patient who has cancer as healthy can lead to more disastrous results (high risk/loss; the life of the patient is at stake) compared to labeling a healthy patient as healthy (low risk/loss; several costly medical tests have been conducted). To incur the least amount of loss/risk, we rationally need to pick the class label that has the smallest expected risk for a given data instance. As can be seen, decision via loss/risk minimization encompasses minimizing labeling error probability as a special case where each $\lambda_{ij} = 1$. Since probability scores can give us an idea about how certain a method is of its labeling decisions, we can add a *reject* decision option (deferring labeling to a human). For instance, if the maximum class probability score is below some threshold, a method can delegate making a labeling decision to a human decider; hence it can reject labeling an instance. Furthermore, the reject decision can have associated risk/loss values and can be included in the risk/loss minimization strategy (if the reject decision attains

the smallest expected risk score, a method can reject the labeling decision). Once the whole joint probability function ($p(\vec{x}, label = l_i)$) is learned, it can be used to generate new data instances by sampling from $p(\vec{x})$ ($p(\vec{x}) = \sum_{i=1}^{K} p(\vec{x}, label = l_i)$). Hence, machine learning methods that estimate $p$(x) (directly or indirectly) are called generative models.

2. Directly learn/estimate the class conditional probability function $p(label|\vec{x})$ from a given dataset, instead of the joint probability function $p(label, \vec{x})$, because the joint probability function contains much more information ($p(\vec{x})$, the likelihood of data instances; for labeling decisions we do not need this information, the conditional probability function suffices). We can utilize the error minimization strategy or the broader strategy of minimizing risk/loss with the estimated class conditional probabilities for labeling new data instances. Machine learning models that estimate the conditional probability scores are coined as discriminative models.

3. Instead of modeling a probability function for labeling, directly learn the labeling function (discriminant function) $f(\vec{x})$ from a given dataset and use $f(\vec{x})$ to label new data instances. Machine learning models such as perceptrons, support vector machines, and decision trees implement a discriminant $f(\vec{x})$. Since these models do not provide a probability score for labeling, we cannot utilize the labeling error minimization or risk/loss minimization strategies. Furthermore, a reject option is not applicable.

Utilization of one of these solution strategies is task/data/budget (time) dependent. All these strategies have their advantages and disadvantages over one another (Bishop chapter 1). Within the scope of this assignment, we are going to consider the second strategy for solving classification problems. We are going to employ multilayer perceptrons (MLPs) to model $p(label|\vec{x})$ and learn the MLP parameters/hyperparameters via MLE.

## 0.2 Solution Strategies for Regression Problems

The solution strategies are quite similar to those of the classification case.

1. Learn/estimate the joint probability function $p(\vec{x}, t)$, where $t$ is a random variable for a numerical value, via a given dataset and use this estimated probability function to obtain the conditional probability $p(t|\vec{x})$ function:

$$P(t|\vec{x}) = \frac{p(\vec{x}, t)}{p(\vec{x})} = \frac{p(\vec{x}, t)}{\int p(x, t)dt}$$

   The conditional probability function does not provide a single value for a given data instance $\vec{x}$ (it provides a distribution over values). To assign a numerical value (to label), we can calculate the expected numerical value for the data instance with the conditional function:

$$\int tp(t|\vec{x})dt$$

2. Directly learn/estimate the class conditional probability function $p(t|\vec{x})$ from a given dataset, instead of the joint probability $p(t, \vec{x})$. Similar to the first strategy, we can calculate the expected value for a given data instance.

3. Instead of modeling a probability function for labeling, directly learn the labeling function (regression function) $f(\vec{x})$ from a given dataset and use $f(\vec{x})$ to label new data instances.

Similar to the classification case, utilization of one of these solution strategies is task/data/budget (time) dependent. All these strategies have their advantages and disadvantages over one another (Bishop Chapter 1). Within the scope of this assignment, we are going to consider the second strategy for solving regression problems. We are going to employ multilayer perceptrons (MLPs) to model $p(t|\vec{x})$ and learn the MLP parameters/hyperparameters via MLE.

# Part 1

Multilayer Perceptrons (MLPs)(feed-forward neural networks) are universal function approximators [1], that is to say, they can approximate any given function ($y = f(x)$; functions to be approximated have to comply with certain requirements, e.g., continuous, bounded functions) arbitrarily closely provided that the MLP has sufficiently many nodes and layers. As a result, we can utilize them to model the probability distribution function $p(label|\vec{x}, \theta)$ for labeling problems (classification and regression). In addition, they can do feature engineering (they find/create/extract/learn the most important features automatically for a given problem) at their hidden layers and possess generalization capabilities (e.g., they can interpolate their results for new unseen data samples).

In this part, we employ MLPs for regression and classification tasks. Figures 1 and 2 depict two MLPs on which the backpropagation update rules should be extracted and implemented. Both networks have 5 layers (the input layer ($x$), first hidden layer ($F$), second hidden layer ($S$), third hidden layer ($T$), and the output layer ($O$)). The subscripts represent the index of the nodes (the order is from top to bottom) in each layer. $x_i$ represents the $i^{\text{th}}$ input to the network whereas $O_n$ is the $n^{\text{th}}$ output of it. For instance, suppose the following input vector is fed to the regression network: $[-0.01, 0.5, 0.9]$ ($x_0 = 1$, $x_1 = 0.5$, $x_2 = 0.2$, $x_3 = 0.9$) (the first component of the input is always +1; it is the input value for the bias parameters of the first hidden layer). After the network performs its calculation at the output, we observe a numeric value (e.g., $O_0 = 0.8$). With the same procedure, we observe a vector at the output of the classification architecture (e.g., $O = [0.4, 0.2, 0.3, 0.1]$, $O_0 = 0.4$, $O_1 = 0.2$, $O_2 = 0.3$, $O_3 = 0.1$). Moreover, for both architectures, $F_j$, $S_k$, and $T_m$ represent the outputs of the hidden layers, first, second, and third, respectively. The first nodes of the hidden layers hold a constant value of 1 for the bias parameters of those layers.

Weights between the input and the first hidden layer are denoted with $A_{ij}$. The index $i$ is the index of a node in the input layer, whereas $j$ is the index of a node in the first hidden layer, so $A_{ij}$ is the weight of the connection from the $i^{th}$ node in the input layer to the $j^{th}$ node in the first hidden layer. With the same naming convention, $B_{jk}$ represents the weights between the first hidden layer and the second hidden layer. Similarly, $C_{km}$ denotes the weights between the second hidden layer and the third hidden layer. Finally, the weights between the third hidden layer and the output layer are denoted by $D_{mn}$. Several weights are explicitly shown in the figures for further clarification.

In both architectures, you are expected to utilize the sigmoid function as the activation function for the hidden layers. For the output of the classification architecture, we employ the softmax function, whereas no function is applied at the output of the regression architecture (in other words, we utilize the identity function $f(x) = x$ for the regression architecture output). With these basics, the following are the operations taking place in the regression architecture:

$$F_j = \sigma(\sum_{i=0}^{3} A_{ij} * x_i)$$

$$S_k = \sigma(\sum_{j=0}^{4} B_{jk} * F_j)$$

$$T_m = \sigma(\sum_{k=0}^{4} C_{km} * S_k)$$

$$O_0 = \sum_{m=0}^{3} D_{mn} * T_m$$

Similarly, the following calculations are for the classification architecture:

$$F_j = \sigma(\sum_{i=0}^{2} A_{ij} * x_i)$$

$$S_k = \sigma(\sum_{j=0}^{5} B_{jk} * F_j)$$

$$T_m = \sigma(\sum_{k=0}^{3} C_{km} * S_k)$$

$$X_n = \sum_{m=0}^{2} D_{mn} * T_m$$

$$O_n = softmax(X_n, X)$$

where

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$$X = [X_0, X_1, X_2, X_3]$$

$$softmax(x, X) = \frac{e^x}{\sum_{s=0}^{3} e^{X_s}}$$

The softmax function requires all output node values to be known for calculations. This is the reason why a new temporary variable ($X_n$) has been introduced. Depending on this temporary variable's values, the softmax function normalizes all output values and converts them into probability scores. The architecture models $p(label|\vec{x}, \theta)$, where $\theta$ includes all the weights of it.

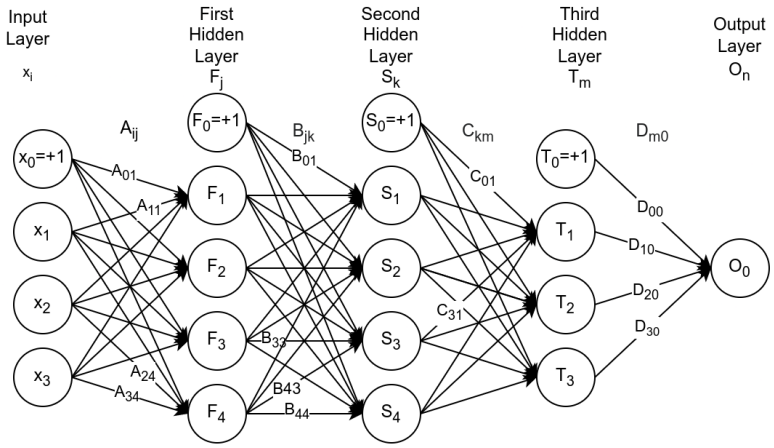



Figure 1: The MLP architecture of Part 1 for the regression problem. $A_{ij}$ denotes the weights between the input and the first hidden layer. Similarly, $B_{jk}$, $C_{km}$, and $D_{m0}$ denote the weights between the first hidden layer and the second hidden layer, the second hidden layer and the third hidden layer, and the third hidden layer and the output layer, respectively

Basically, we want a neural network to learn from the information we provide and make guesses about previously unseen information. Hopefully, it yields correct results for this unknown information. To assess whether the network learns or not, we can easily check the output of the network and the ground truth for the information. So, we want it to output values/guesses as closely as possible to the ground truth. We can define a function that measures the closeness between the truth and the predicted values by the neural network. If they are close, the function can generate lower values; otherwise, higher values. So we can cast this neural network learning problem as a function optimization problem. In this respect, we alter the network weights in such a way that a particular function is minimized. Here, minimization corresponds to bringing the network outputs as close as possible to the actual truth. From calculus, we simply know that to minimize a function, we should take steps in the reverse direction of the gradient vector. The

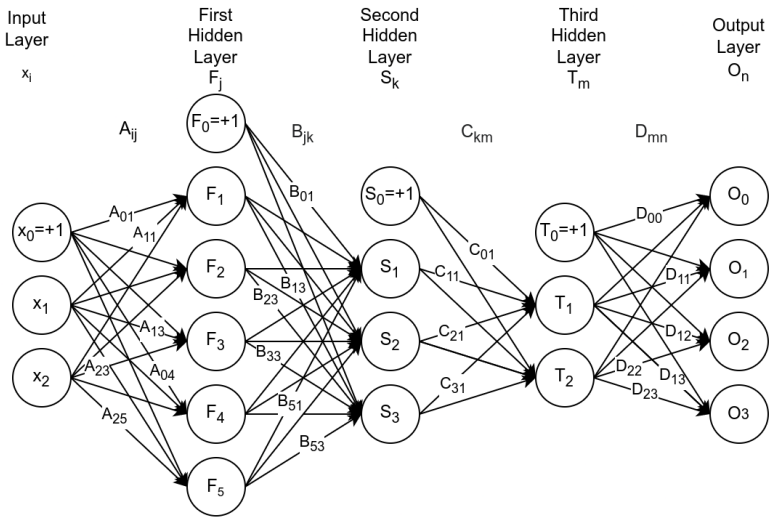



Figure 2: The MLP architecture of Part 1 for the classification problem. Identical to the regression architecture, $A_{ij}$ denotes the weights between the input and the first hidden layer. Similarly, $B_{jk}$, $C_{km}$, and $D_{mn}$ denote the weights between the first hidden layer and the second hidden layer, the second hidden layer and the third hidden layer, and the third hidden layer and the output layer, respectively

backpropagation algorithm basically applies this logic; it is the direct application of the gradient-descent method for minimizing particular loss functions (e.g., loss functions should be differentiable to apply the backpropagation algorithm). So for the regression network, we can utilize the squared error (SE) function, which has the following definition:

$$SE(y, y') = (y - y')^2$$

where $y$ is the ground truth and $y'$ is the output of the network for a particular input (e.g., $\vec{x}$). For our setting, this function can be written as:

$$SE(y, O_0) = (y - O_0)^2$$

Similarly, we can utilize the cross-entropy function for the classification architecture, and it has the following form:

$$CE(l, O) = -\sum_n l_n * log(O_n)$$

where $l$ stores the actual ground truth for the classification task, and $O$ is the output of the network ($log$ is the natural logarithm function, $log_e$). The classification task requires more outputs since each output provides a probability score for each different class in the classification problem. Mathematically, the first output node models/outputs $P(C_1|\vec{x}, \theta)$, the second output node models/outputs $P(C_2|\vec{x}, \theta)$, the third output node models/outputs $P(C_3|\vec{x}, \theta)$, and so forth, where $\vec{x}$ is an input data point, each $C_i$ represents a particular class label, and $\theta$ denotes the network's parameters (weights and biases). The $softmax$ function converts the raw output values of the network into probability scores. As we can easily see, the cross-entropy function works on vectors (not single values), and it aims to raise the probability score generated by the network for the correct class. In a classification problem of four classes, each class can be represented with the following vector: [1.0, 0.0, 0.0, 0.0] (for the first class), [0.0, 1.0, 0.0, 0.0] (for the second class), [0.0, 0.0, 1.0, 0.0] (for the third class) and [0.0, 0.0, 0.0, 1.0] (for the fourth class), which is called one-hot encoding. And the network outputs four values ($[O_0, O_1, O_2, O_3]$); hence, the cross-entropy function for our setting is as follows:

$$CE(l = [l_0, l_1, l_2, l_3], O = [O_0, O_1, O_2, O_3]) = -\sum_{n=0}^{3} l_n * log(O_n)$$

In this part, you are expected to derive update rules for $A_{ij}$, $B_{jk}$, $C_{km}$ and $D_{mn}$ in detail for both regression and classification architectures separately by applying the backpropagation algorithm. Later, you are expected to implement these update rules for the given regression and classification tasks.

The generic weight update rule is as follows:

$$w^{new} = w - \eta * \frac{\partial E(x)}{\partial w}$$

where $\eta$ is the learning rate and $E$ is the function to be minimized with respect to $w$.

## Part 1 Specifications

- You are expected to show/derive each weight update rule in detail and in a step-by-step fashion by utilizing the backpropagation algorithm for both regression and classification problems represented here.

- For regression, please assume that you are given a single ground truth $y$ and its input $[x_1, x_2, x_3]$ ($x_0 = 1$), and your derivation calculations should involve these values. So the following are the update rules for the regression problem (you are expected to further expand these equations **in detail** by employing the backpropagation algorithm):

$$A_{ij}^{new} = A_{ij} - \eta \frac{\partial SE(y, O_0)}{\partial A_{ij}}$$

$$B_{jk}^{new} = B_{jk} - \eta \frac{\partial SE(y, O_0)}{\partial B_{jk}}$$

$$C_{km}^{new} = C_{km} - \eta \frac{\partial SE(y, O_0)}{\partial C_{km}}$$

$$D_{m0}^{new} = D_{m0} - \eta \frac{\partial SE(y, O_0)}{\partial D_{m0}}$$

- Similarly, for classification, assume that you are given a single ground truth $[l_0, l_1, l_2, l_3]$ (in the one-hot encoding format, $\sum_n l_n = 1$) and its input vector $[x_1, x_2]$, and your derivation calculations should involve these values. So the following updates are for the classification problem (you are expected to further expand these equations **in detail** by employing the backpropagation algorithm):

$$A_{ij}^{new} = A_{ij} - \eta \frac{\partial CE([l_0, l_1, l_2, l_3], O = [O_0, O_1, O_2, O_3])}{\partial A_{ij}}$$

$$B_{jk}^{new} = B_{jk} - \eta \frac{\partial CE([l_0, l_1, l_2, l_3], O = [O_0, O_1, O_2, O_3])}{\partial B_{jk}}$$

$$C_{km}^{new} = C_{km} - \eta \frac{\partial CE([l_0, l_1, l_2, l_3], O = [O_0, O_1, O_2, O_3])}{\partial C_{km}}$$

$$D_{mn}^{new} = D_{mn} - \eta \frac{\partial CE([l_0, l_1, l_2, l_3], O = [O_0, O_1, O_2, O_3])}{\partial D_{mn}}$$

- You are expected to show/derive the update rules for each problem separately (regression, classification). You are expected to prepare your derivation solutions using LaTeX.

- Your solutions are going to be inspected manually, so please make sure that your solution steps are easy to follow and your writing is legible; if need be, please add explanations for the calculations.
- Instead of providing each single weight update one by one (e.g., $D_{00}^{new}$, $D_{32}^{new}$), you can provide a single update rule for all the weights between layers (e.g., $A_{ij}^{new} = ...$, $B_{jk}^{new} = ...$, $C_{km}^{new} = ...$, $D_{mn}^{new} = ...$) to save effort and time.
- In addition to the manual derivations, this part requires you to implement these backpropagation update rules for solving a regression task and a classification task with the MLP architectures described above.
- For the regression architecture implementation, you are given a file (**mlp_regression_backpropagation.py**) that loads a regression problem dataset and implements a class called `MLPRegressor`. Using the class and data, it trains the architecture via backpropagation and prints performance results during training. Within this class, you are expected to implement the `forward` function that runs the forward propagation operation and the weight update part in the `train` function. The class implements the regression architecture depicted in Figure 1. To simplify numerical calculations, the class separates the weights and biases associated with each layer (weight matrices are decomposed into two). In other words, in the class implementation, eight variables are utilized to store weights: `A`, `A_bias`, `B`, `B_bias`, `C`, `C_bias`, `D`, and `D_bias`, whose definitions are as follows:

$$\texttt{A} = \begin{bmatrix} A_{11} & A_{12} & A_{13} & A_{14} \\ A_{21} & A_{22} & A_{23} & A_{24} \\ A_{31} & A_{32} & A_{33} & A_{34} \end{bmatrix}, \texttt{A_bias} = \begin{bmatrix} A_{01} & A_{02} & A_{03} & A_{04} \end{bmatrix}$$

$$\texttt{B} = \begin{bmatrix} B_{11} & B_{12} & B_{13} & B_{14} \\ B_{21} & B_{22} & B_{23} & B_{24} \\ B_{31} & B_{32} & B_{33} & B_{34} \\ B_{41} & B_{42} & B_{43} & B_{44} \end{bmatrix}, \texttt{B_bias} = \begin{bmatrix} B_{01} & B_{02} & B_{03} & B_{04} \end{bmatrix}$$

$$\texttt{C} = \begin{bmatrix} C_{11} & C_{12} & C_{13} \\ C_{21} & C_{22} & C_{23} \\ C_{31} & C_{32} & C_{33} \\ C_{41} & C_{42} & C_{43} \end{bmatrix}, \texttt{C_bias} = \begin{bmatrix} C_{01} & C_{02} & C_{03} \end{bmatrix}$$

$$\texttt{D} = \begin{bmatrix} D_{10} \\ D_{20} \\ D_{30} \end{bmatrix}, \texttt{D_bias} = \begin{bmatrix} D_{00} \end{bmatrix}$$

The class loads initial weight values from a file so that once you correctly implement the mentioned parts within the class (forward function and weight updates), your training output should be **exactly** the same as the following:

```
Epoch Number: 1 - Training Mean SE: 2.22116
Epoch Number: 2 - Training Mean SE: 2.08498
Epoch Number: 3 - Training Mean SE: 2.02369
Epoch Number: 4 - Training Mean SE: 2.00509
Epoch Number: 5 - Training Mean SE: 1.99682
Epoch Number: 6 - Training Mean SE: 1.99059
Epoch Number: 7 - Training Mean SE: 1.98344
Epoch Number: 8 - Training Mean SE: 2.05438
Epoch Number: 9 - Training Mean SE: 2.00374
Epoch Number: 10 - Training Mean SE: 1.99463
```

```
...
Epoch Number: 42 - Training Mean SE: 1.93658
Epoch Number: 43 - Training Mean SE: 1.93648
Epoch Number: 44 - Training Mean SE: 1.93672
Epoch Number: 45 - Training Mean SE: 1.93655
Epoch Number: 46 - Training Mean SE: 1.93641
Epoch Number: 47 - Training Mean SE: 1.93610
Epoch Number: 48 - Training Mean SE: 1.93578
Epoch Number: 49 - Training Mean SE: 1.93541
Epoch Number: 50 - Training Mean SE: 1.93501
```

- For the classification architecture implementation, you are given an implementation file (**mlp_classification_backpropagation.py**) that loads a classification problem dataset and implements a class called `MLPClassifier`. Using the class and data, it trains the architecture via backpropagation and prints performance results during training. Within this class, you are expected to implement the `forward` function that runs the forward propagation operation and the weight update part in the `train` function. The class implements the classification architecture depicted in Figure 2. Similar to the regression implementation, the class separates the weights and biases associated with each layer and utilizes six variables to store weights: `A`, `A_bias`, `B`, `B_bias`, `C`, `C_bias`, `D`, and `D_bias`, whose definitions are as follows:

$$\texttt{A} = \begin{bmatrix} A_{11} & A_{12} & A_{13} & A_{14} & A_{15} \\ A_{21} & A_{22} & A_{23} & A_{24} & A_{25} \end{bmatrix}, \texttt{A_bias} = \begin{bmatrix} A_{01} & A_{02} & A_{03} & A_{04} & A_{05} \end{bmatrix}$$

$$\texttt{B} = \begin{bmatrix} B_{11} & B_{12} & B_{13} \\ B_{21} & B_{22} & B_{23} \\ B_{31} & B_{32} & B_{33} \\ B_{41} & B_{42} & B_{43} \\ B_{51} & B_{52} & B_{53} \end{bmatrix}, \texttt{B_bias} = \begin{bmatrix} B_{01} & B_{02} & B_{03} \end{bmatrix}$$

$$\texttt{C} = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \\ C_{31} & C_{32} \end{bmatrix}, \texttt{C_bias} = \begin{bmatrix} C_{01} & C_{02} \end{bmatrix}$$

$$\texttt{D} = \begin{bmatrix} D_{10} & D_{11} & D_{12} & D_{13} \\ D_{20} & D_{21} & D_{22} & D_{23} \end{bmatrix}, \texttt{D_bias} = \begin{bmatrix} D_{00} & D_{01} & D_{02} & D_{03} \end{bmatrix}$$

  The class loads initial weight values from a file so that once you correctly implement the mentioned parts within the class (forward function and weight updates), your training output should be **exactly** the same as the following:

```
Epoch Number: 1 - Training Mean CE: 1.386 - Training Accuracy: 25.00
Epoch Number: 2 - Training Mean CE: 1.384 - Training Accuracy: 25.00
Epoch Number: 3 - Training Mean CE: 1.382 - Training Accuracy: 25.00
Epoch Number: 4 - Training Mean CE: 1.380 - Training Accuracy: 25.00
Epoch Number: 5 - Training Mean CE: 1.377 - Training Accuracy: 25.00
Epoch Number: 6 - Training Mean CE: 1.373 - Training Accuracy: 25.00
Epoch Number: 7 - Training Mean CE: 1.367 - Training Accuracy: 25.00
Epoch Number: 8 - Training Mean CE: 1.357 - Training Accuracy: 25.00
Epoch Number: 9 - Training Mean CE: 1.341 - Training Accuracy: 28.70
Epoch Number: 10 - Training Mean CE: 1.314 - Training Accuracy: 51.20
Epoch Number: 11 - Training Mean CE: 1.268 - Training Accuracy: 71.40
...
Epoch Number: 46 - Training Mean CE: 0.104 - Training Accuracy: 99.60
Epoch Number: 47 - Training Mean CE: 0.099 - Training Accuracy: 99.60
```

```
Epoch Number: 48 - Training Mean CE: 0.096 - Training Accuracy: 99.60
Epoch Number: 49 - Training Mean CE: 0.092 - Training Accuracy: 99.60
Epoch Number: 50 - Training Mean CE: 0.089 - Training Accuracy: 99.60
```

# Part 2

In this part, you are expected to implement two neural network architectures for one classification task and one regression task, respectively, using the basic operations of PyTorch (no high-level features such as custom modules or PyTorch modules/layers are allowed). The classification task of this part involves learning to predict class labels (out of 3 classes) for a given unseen data sample, whereas the regression task requires learning to predict a single numeric value.

To this end, for the classification task, you are expected to implement a multilayer perceptron which consists of 1 input layer (with 8 nodes), 3 hidden layers (with 64, 32, 16 nodes, respectively), and 1 output layer (with 3 nodes). The hidden layers are expected to apply the leaky ReLU function, and the output should apply the softmax function. The softmax function (with the 3 output units) is utilized to calculate the posterior class distribution for a given data sample $\vec{x}$ (e.g., $P(C|\vec{x})$ = [class1 probability score, class2 probability score, class3 probability score]). In order to predict the label of $\vec{x}$, we can choose the class whose probability score is the highest among the others in order to minimize the error probability of making a wrong labeling decision. For instance, if the network outputs the following scores for $\vec{x}$, [class1=0.5, class2=0.3, class3=0.2], we pick class1 as the label of $\vec{x}$ (when we pick class1 as the label, we have an error probability value of 0.5; on the other hand, with class2 and class3 we have error probability values of 0.7 and 0.8, respectively, which are higher than 0.5). Since it is a multiclass classification task, the network can be trained with the cross-entropy loss function, which is defined as follows:

$$CrossEntropy(groundtruth = l, predicted = p) = -\sum_i l_i * log(p_i)$$

where $l_i$ and $p_i$ represent the i$^{th}$ component of the true label and the predicted distribution, respectively. For instance, suppose that the label vector is $[1, 0, 0]$ (the instance is of type class 1) and the network has yielded $[0.2, 0.2, 0.6]$ (the network predicts class3 to be the most probable class for $\vec{x}$). So the cross entropy loss ($CrossEntropy(predicted = [0.2, 0.2, 0.6], groundtruth = [1, 0, 0])$) is $-(1 * \log(0.2) + 0 * log(0.2) + 0 * \log(0.6)) = 1.609$. The network aims to learn the class distribution of the training data by minimizing the cross-entropy loss and to classify unseen data (by hoping that the training data distribution represents the unseen data distribution (is rich enough to generalize)).

For the regression task, you are expected to implement a multilayer perceptron that consists of 1 input layer (7 nodes), 3 hidden layers (with 256, 128, 64 nodes, respectively), and 1 output layer (with 1 node). The hidden layers are expected to apply the SELU function. The output layer does not apply any activation function. The output yields a single numeric value for a given data sample. The network is trained with the squared error, which is defined as follows:

$$SE(groundtruth = y, predicted = y') = (y - y')^2$$

For instance, if the network outputs 0.5 for a data sample $\vec{x}$ and the ground truth of $\vec{x}$ is 1.5, the squared error score becomes $(1.5 - 0.5)^2 = 1.0$. The network aims to learn the underlying labeling function of the training data by minimizing the squared error loss and to predict numeric values of unseen data (by hoping that the training data distribution is capable of representing the unseen data distribution).

So far, we have considered only a single data instance to update network weights. In practice, we come across three main approaches: stochastic gradient descent learning, batch gradient descent learning, and mini-batch gradient descent learning. In stochastic gradient descent learning, all the network weights are updated via a single instance. A single instance is fed to the network, and backpropagation is executed for the incurred cost due to this single instance (the same procedure is repeated for other data instances).

In batch learning, all data instances are used to update the weights in one go. All instances are fed to the network, and the backpropagation algorithm is executed with the error incurred due to all instances. The minibatch gradient ascent learning stands between the other two. A specific number of instances are utilized for the weight updates (not a single instance nor all instances). The data set is divided into groups of fixed size. A group of instances is fed to the network, and the backpropagation algorithm is executed over the loss incurred due to this group (the next step, another group is picked, and this procedure is continued). For this part, you are expected to consider only batch gradient descent learning. To this end, **we are going to utilize average/mean cross-entropy loss and average/mean squared error for training the networks mentioned above**. Averaging error enables us to compare a method's performance on datasets with a different number of instances (e.g., it is useful for forming the SRM plot). In addition, it can provide insights into a method's average performance on a dataset.
To clarify average cross-entropy loss and squared error loss calculations, consider the following datasets and ground truths for a classification and a regression task. Each row vector represents a single data sample (in $X_i$, each $x_i^j$ is the $i^{\text{th}}$ feature value of the $j^{th}$ instance) and its corresponding label (in $L_i$).

$$X_1 = \begin{bmatrix}\text{instance 1}\\ \text{instance 2}\\ \text{instance 3}\\ \text{instance 4}\end{bmatrix} = \begin{bmatrix}x_1^1 & x_2^1 & x_3^1\\ x_1^2 & x_2^2 & x_3^2\\ x_1^3 & x_2^3 & x_3^3\\ x_1^4 & x_2^4 & x_3^4\end{bmatrix}, L_1 = \begin{bmatrix}\text{label of instance 1}\\ \text{label of instance 2}\\ \text{label of instance 3}\\ \text{label of instance 4}\end{bmatrix} = \begin{bmatrix}1 & 0 & 0\\ 0 & 1 & 0\\ 0 & 0 & 1\\ 1 & 0 & 0\end{bmatrix}$$

$$X_2 = \begin{bmatrix}\text{instance 1}\\ \text{instance 2}\\ \text{instance 3}\end{bmatrix} = \begin{bmatrix}a_1^1 & a_2^1\\ a_1^2 & a_2^2\\ a_1^3 & a_2^3\end{bmatrix}, L_2 = \begin{bmatrix}\text{label of instance 1}\\ \text{label of instance 2}\\ \text{label of instance 3}\end{bmatrix} = \begin{bmatrix}2.0\\ 1.5\\ -0.3\end{bmatrix}$$

$x_i^j, a_k^l \in \mathbb{R}$. Suppose that we have constructed an MLP ($MLP_1$) for the classification task and another MLP ($MLP_2$) for the regression task, and when we feed the datasets to them, we obtain the following outputs, respectively:

$$MLP_1(X_1) = \begin{bmatrix}MLP_1(\text{instance 1})\\ MLP_1(\text{instance 2})\\ MLP_1(\text{instance 3})\\ MLP_1(\text{instance 4})\end{bmatrix} = \begin{bmatrix}0.1 & 0.8 & 0.1\\ 0.7 & 0.2 & 0.1\\ 0.2 & 0.25 & 0.55\\ 0.05 & 0.93 & 0.02\end{bmatrix}$$

$$MLP_2(X_2) = \begin{bmatrix}MLP_2(\text{instance 1})\\ MLP_2(\text{instance 2})\\ MLP_2(\text{instance 3})\end{bmatrix} = \begin{bmatrix}0.1\\ -2.0\\ 1.1\end{bmatrix}$$

With these neural network outputs and ground truths, we can calculate the average cross-entropy (for the classification task) and the average mean squared error loss (for the regression task) as follows:

$$CrossEntropy(l = L_1, p = MLP_1(X_1)) = \begin{bmatrix}-(1*\log(0.1)+0*\log(0.8)+0*\log(0.1))\\ -(0*\log(0.7)+1*\log(0.2)+0*\log(0.1))\\ -(0*\log(0.2)+0*\log(0.25)+1*\log(0.55))\\ -(1*\log(0.05)+0*\log(0.93)+0*\log(0.02))\end{bmatrix} = \begin{bmatrix}2.30\\ 1.61\\ 0.60\\ 3.00\end{bmatrix}$$

$$SE(L_2, MLP_2(X_2)) = \begin{bmatrix}(2.0-0.1)^2\\ (1.5-(-2.0))^2\\ (-0.3-1.1)^2\end{bmatrix} = \begin{bmatrix}3.61\\ 12.25\\ 1.96\end{bmatrix}$$

$$MeanCrossEntropy(l = L_1, p = MLP_1(X_1)) = Mean(\begin{bmatrix}2.30\\ 1.61\\ 0.60\\ 3.00\end{bmatrix}) = \begin{bmatrix}1.88\end{bmatrix}$$

$$MeanSE(L_2, MLP_2(X_2)) = Mean(\begin{bmatrix}3.61\\ 12.25\\ 1.96\end{bmatrix}) = \begin{bmatrix}5.95\end{bmatrix}$$

## Part 2 Specificiations

- The classification task involves the classification of data samples into three categories: class1, class2, and class3. Each data sample consists of 8 real-valued features. The labels (ground truth) of the data samples are represented with the one-hot vector encoding mechanism. The class1, class2, and class3 samples are represented with $[1, 0, 0], [0, 1, 0], [0, 0, 1]$, respectively.
- The regression task of this part requires predicting a numerical value for data samples. Each sample consists of 7 real-numbered features, and the labels are single real numbers.
- You have been provided with two .py files (`part2_mlpclassification.py`, `part2_mlpregression.py`) and are expected to complete the missing parts (the `forward()` function and other parts marked via '...') by using solely the basic operations of PyTorch (e.g torch.matmul, torch.argmax, torch.sigmoid, torch.from_numpy .etc). The files contain helper comments for details.
- For each task, three datasets are provided: training, validation, and test. The neural networks are trained on the training datasets. At each training step, neural networks' loss scores (mean cross-entropy loss for the classification task, mean squared error for the regression task) are measured both on the training datasets and validation sets. For the classification task, as another metric, the accuracy scores are calculated. When the training operation is completed, the performance of the trained network is measured on the test dataset for each task type.
- The accuracy performance metric is defined as follows:

$$Accuracy(predicted, groundtruth) = \frac{\text{\# of data instances whose labels are predicted correctly}}{\text{\# of instances}} * 100$$

- The implementations train the networks with the gradient descent algorithm with a fixed number of iterations by using all dataset instances (batch gradient descent learning). At the end of the training, the history of training dataset loss versus validation dataset loss is plotted (the SRM plot).
- This part aims to familiarize you with the basic working principle of PyTorch along with its basic operations. You don't need to perform any hyperparameter tuning or search in this part, which is the main consideration of the third part.
- As a reference, the following outputs are obtained at the end of the executions of the completed implementations.

```
Iteration : 992 - Train Loss 0.3317 - Train Accuracy : 88.03 - Validation Loss : 0.4965 Validation Accuracy : 78.93
Iteration : 993 - Train Loss 0.3314 - Train Accuracy : 88.04 - Validation Loss : 0.4966 Validation Accuracy : 78.92
Iteration : 994 - Train Loss 0.3312 - Train Accuracy : 88.05 - Validation Loss : 0.4966 Validation Accuracy : 78.90
Iteration : 995 - Train Loss 0.3309 - Train Accuracy : 88.06 - Validation Loss : 0.4967 Validation Accuracy : 78.88
Iteration : 996 - Train Loss 0.3306 - Train Accuracy : 88.07 - Validation Loss : 0.4968 Validation Accuracy : 78.89
Iteration : 997 - Train Loss 0.3304 - Train Accuracy : 88.08 - Validation Loss : 0.4968 Validation Accuracy : 78.89
Iteration : 998 - Train Loss 0.3301 - Train Accuracy : 88.09 - Validation Loss : 0.4969 Validation Accuracy : 78.90
Iteration : 999 - Train Loss 0.3298 - Train Accuracy : 88.11 - Validation Loss : 0.4970 Validation Accuracy : 78.89
Iteration : 1000 - Train Loss 0.3296 - Train Accuracy : 88.12 - Validation Loss : 0.4970 Validation Accuracy : 78.88
Test accuracy : 78.29
```

Figure 3: A sample output for the classification task implementation

```
Iteration : 1491 - Train MSE Loss 14.7705 - Validation MSE Loss : 18.87
Iteration : 1492 - Train MSE Loss 14.8269 - Validation MSE Loss : 18.16
Iteration : 1493 - Train MSE Loss 14.8745 - Validation MSE Loss : 19.01
Iteration : 1494 - Train MSE Loss 14.9408 - Validation MSE Loss : 18.26
Iteration : 1495 - Train MSE Loss 14.9942 - Validation MSE Loss : 19.16
Iteration : 1496 - Train MSE Loss 15.0703 - Validation MSE Loss : 18.37
Iteration : 1497 - Train MSE Loss 15.1290 - Validation MSE Loss : 19.32
Iteration : 1498 - Train MSE Loss 15.2154 - Validation MSE Loss : 18.49
Iteration : 1499 - Train MSE Loss 15.2841 - Validation MSE Loss : 19.50
Iteration : 1500 - Train MSE Loss 15.3746 - Validation MSE Loss : 18.63
Test MSE loss : 21.4682
Test RMSE loss : 4.6334
```

Figure 4: A sample output for the regression task implementation

**Note:** You are not expected to obtain the same outputs shown here since the results printed are likely to differ from run to run for MLPs, since weights are initialized randomly at the beginning of each run (if we do not take any measures against it).

# Part 3

Almost every machine learning method/algorithm possesses one or more hyperparameters that need to be tuned and are very crucial for the method to work properly. Multilayer perceptrons are no exception. First of all, a structure (network topology) must be determined. The input and output layer sizes are determined by the problem to be solved, but other factors such as how many hidden layers there should be, how many neuron units each of these hidden layers should contain, what type of activation functions (sigmoid, tanh, ReLU, leaky ReLU, etc.) should be employed on these units, whether a neuron should be connected to all neuron units in the previous layer, etc., need to be determined before attempting to solve the given problem. Since we need to employ the gradient descent algorithm (via backpropagation), the learning rate, which controls the amount of change in each weight of the network, and the number of times that the weights are updated (iterations, number of epochs) should be decided. All these (and more) constitute the hyperparameters of an MLP. Within the scope of this part, the following are the hyperparameters of an MLP: number of hidden layers, number of units in a hidden layer or hidden layers, learning rate, the number of epochs (number of iterations), the activation function utilized in a neuron unit.

To determine which hyperparameter values are the most suitable for a given problem (the optimal hyperparameter values are task-dependent), we need to perform a hyperparameter search. In a typical machine learning problem setting, we have a training dataset and a test dataset. The training dataset is used to fine-tune the parameters (e.g., neural network weights) of the algorithm, whereas the test dataset is used to assess the generalization performance of the model/algorithm (after training). Each hyperparameter value defines a different instance of a model/algorithm family (e.g MLP with 2 hidden layers, MLP with 1 hidden layer, since they can restrain or expand the function space that an algorithm searches through during learning; please recall MLPs are function approximators). So different hyperparameters result in different instances of the same model/algorithm. Each of these model instances may have different generalization capabilities. To compare their generalization capabilities, we can not use the training dataset, since a group of MLPs with different hyperparameter values can easily memorize the training dataset (overfitting). Hence, they can be easily thought to perform equally well, which is a very critical mistake with regard to generalization. The test dataset can not be used either because they can easily overfit to the test set too (this time, in addition to the parameters, the hyperparameters can be tuned so as to perform well on the test set, which is another methodological mistake). Instead, along with the training dataset and test dataset, we can have a validation dataset. On this dataset, we can test the generalization capabilities of different hyperparameters. Even if different hyperparameters overfit the training data, their performance on the validation dataset provides a useful idea for generalization. By tuning the hyperparameters with respect to the validation dataset, this time, the model overfits the validation dataset, but we have another dataset (test data) to measure the unbiased generalization performance. In summary, we utilize a training set to tune the parameters of a model/algorithm, a validation dataset to fine-tune the hyperparameters of the model/algorithm, and a test set to assess the unbiased expected generalization performance of these fine-tuned parameters and hyperparameters.

During a hyperparameter search, we can fine-tune hyperparameters one at a time. For instance, suppose that we have three hyperparameters: A, B, and C. First, we can set B and C to some fixed values and alter A's values to determine the best value for A depending on the performance on a validation dataset. Later, we set A to its best value that has been found and C to a fixed value, then alter B's value. While searching for C, we can use the best values of A and B and then alter C values. This type of strategy may be useful for some algorithms, but it lacks the ability to capture the relationship between hyperparameter values; hence, it is not a generic strategy to consider. A better approach is

to test all possible combinations (Cartesian product) of hyperparameter values, which is coined as grid search. For instance, we decide the following hyperparameters and their values: the number of neuron units in a hidden layer (referred to as number of neurons) = $\{10, 30\}$, learning rate = $\{0.001, 0.0001\}$, number of epochs = $\{150, 250\}$. The grid search tests all the following hyperparameter configurations (combinations):

| Number of Neurons | Learning Rate | Number of Epochs |
|---|---|---|
| 10 | 0.001 | 150 |
| 10 | 0.001 | 250 |
| 10 | 0.0001 | 150 |
| 10 | 0.0001 | 250 |
| 30 | 0.001 | 150 |
| 30 | 0.001 | 250 |
| 30 | 0.0001 | 150 |
| 30 | 0.0001 | 250 |

Table 1: A sample grid-search setting with three hyperparameters

In this part, you are expected to perform an extensive hyperparameter search with the MLP algorithm/method for a classification task and report performance results.

Reproducibility and statistical significance of reported results are of great importance for a machine learning application/study. Especially if there is stochasticity (randomness) at any step of a machine learning study/application/project (e.g randomness due to the method considered, randomness due to data loading, randomness while selecting instances during training, etc.), the final results reported after a single run may be a fluke and bear no, unfortunately, statistical significance, which may lead to very inaccurate and deceptive interpretations, especially while comparing different methods/algorithms over a particular ML task. As a result, the results are highly likely not to be reproducible, which especially hinders and harms advancements in machine learning research. MLPs are inherently stochastic since their weights are initialized randomly at each execution. In addition, other factors can further amplify randomness, such as the usage of stochastic gradient descent (where a single instance is used to update the weights), the order of instances considered, and shuffling of the dataset instances, etc. The reader is encouraged to refer to the reproducibility page [2] of PyTorch for further details regarding the randomness sources. To alleviate the effects of these randomness sources on the results measured (e.g., performance results change from run to run), we can run methods multiple times and calculate a statistical measure that can provide an idea about the distribution of the attained performance results. To this end, in this part we are going to employ the confidence interval (CI) measure (95%) which is defined as follows (Alpaydın Chapter 20):

$$\text{Confidence Interval}(s_1, s_2, s_3, \ldots, s_N) = \mu \pm 1.96 * \frac{\sigma}{\sqrt{N}}$$

where $s_i$s are measured performance values for which the confidence interval is calculated, $N$ is the number of $s_i$s, $\mu$ and $\sigma$ are the average and standard deviation of $s_i$s, respectively. This measure simply calculates the interval probability for the true average performance score: $p(\mu - 1.96 * \frac{\sigma}{\sqrt{N}} \leq$ true average method performance $\leq \mu + 1.96 * \frac{\sigma}{\sqrt{N}}) = 0.95$. So when applied to method performance scores, it states that if we repeat experiments and calculate intervals with new $s_i$s, 95% of these calculated intervals are expected to contain a model's true average performance score. Hence, it can provide statistical confidence as to how a method performs on average in a given problem. In addition, the width of the interval can give an idea about how robust a method is for a given problem, since if a method yields almost identical results for each performance measurement, its average performance confidence interval will be very narrow due to the variance being very small.

## Part 3 Specifications

- This part asks you to both implement an MLP training code for a classification task and prepare a report (in LaTeX) about the training setting, procedures you have considered, and the decisions made throughout the process, as well as the results attained.
- You have been provided with three datasets (train, validation, test) that have been obtained from butterflow/moth images and the initial source code file (`part3.py`) that loads these datasets. In this part, only a single classification task is considered: identification of butterfly/moth species from RGB images.
- In order to solve this problem, you are expected to work and implement solely multilayer perceptrons by utilizing higher-level PyTorch features (e.g., loss functions, custom modules, etc.).
- You are expected to perform an extensive parameter search (grid search) over the MLP hyperparameters (number of hidden layers, number of neurons in these layers, learning rate, number of iterations (epochs), activation functions) (You are free to expand this list, but these are the necessary hyperparameters for this part). During hyperparameter search, each hyperparameter configuration should be executed 10 times, and their confidence interval scores for the accuracy metric should be reported. After the hyperparameter search (determining the best hyperparameter values with respect to these interval scores (e.g., picking the configuration that attains the highest mean accuracy score)), the validation dataset and the training sets should be combined into a larger dataset. The best hyperparameter setting should be employed on this new training dataset, and the accuracy score on the test dataset should be measured. Again, this operation should be repeated 10 times, and a confidence interval for the accuracy performance score on the test dataset should be reported.
- You are free to choose which hyperparameter values to test (please state them in your report clearly) and the report format. You can utilize the Adam optimizer rather than the SGD optimizer of PyTorch. If you know or have devised a way to eliminate the search over the number of iterations (epochs) parameter, you don't need to perform a hyperparameter search over it. If so, please explain how you have achieved eliminating it in your report.
- You may consider different types of activation functions rather than sigmoid and tanh (or you can test only these two). You may refer to the PyTorch documentation for other activation functions.
- During training, you can prefer to use batch gradient descent learning or minibatch gradient descent learning. If you consider choosing the second way, the minibatch size becomes another hyperparameter whose optimal value should be determined via the grid search.
- If you would like, you can save/load your trained models via `torch.save` and `torch.load`. In addition, if you would like to store temporarily or make a copy of the current parameter values of the network, you can use the following code snippet:

```
import copy
...
...
stored_weights = copy.deepcopy(model.state_dict()) # saving current weight
    values
...
...
...
# restore the previously saved weight values
# the current weight values of the model are overwritten with
   stored_weights
model.load_state_dict(stored_weights)
```

- You are expected to test at least 10 hyperparameter configurations for this part. You can provide SRM plots (by taking the average of both train and validation losses over 10 runs) for the hyperparameter search procedure. The hyperparameters and the values that you have tried should be visible in your implementations.
- You can provide a single source code that performs all the work of this part in a single run or provide separate Python files for each different hyperparameter configuration. If you consider the second way, please compress all these source code files into a single file during the submission.
- The input layer of your MLP design should consist of 30000 units (butterfly/moth images are 100x100 RGB images, and each image has been flattened to a vector of size 100*100*3=30000 in order to be fed to the network as an input). As for the output layer size, 100 must be considered since there are 100 different species. Figure 5 shows sample images from different species categories in the problem.

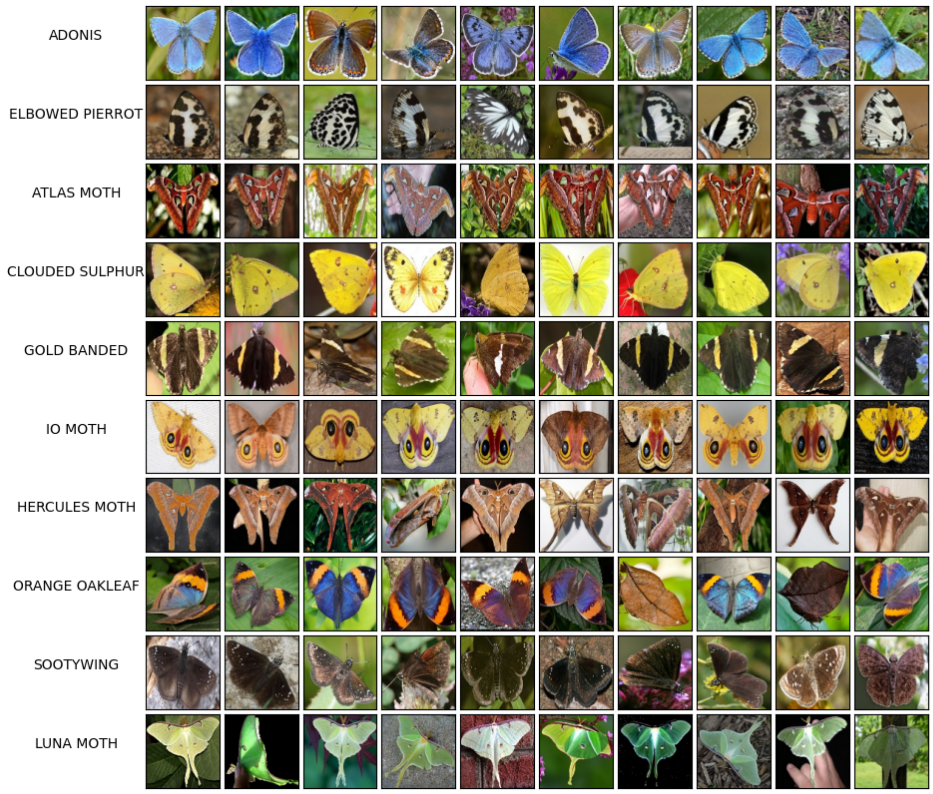



Figure 5: Sample images from the dataset

- In PyTorch, there is more than one way to implement and train via the cross-entropy loss function. If you consider using **torch.nn.CrossEntropyLoss**, you should not apply the softmax function at the output layer, since this function already considers softmax internally. In this case, your model implementation is going to provide raw output values (without the softmax function) when you try to predict the label of an instance. To be able to obtain probability distribution scores, you should explicitly apply the softmax function, especially when checking the accuracy performance on the training, validation, and test datasets. To this end, the following code snippet can be utilized:

```
...
softmax_function = nn.Softmax(dim=1)
...
```

```
nn_output = model(training_dataset) # model does not apply the softmax
    function due to nn.CrossEntropyLoss
loss = loss_function(nn_output, groundtruth)
...
...
probability_scores = softmax_function(nn_output) # we explicitly apply the
    softmax function to get probability scores
...
```

- You don't need to set any seed values for PyTorch or Numpy at the very beginning of source files. In this part, we attempt to alleviate the effect of the randomness emerging by calculating a statistical metric over the results attained.
- Please add explanatory comments on your implementation since all the work is going to be graded manually. Please, in your report, explain your training setup (hyperparameters, values, neural network architecture, etc.), procedure (how you have trained your model, the accuracy performance results of hyperparameter configurations on the validation dataset, etc.), and the final test set accuracy results (all performance score confidence intervals on all datasets should be provided explicitly).
- Additionally, please answer the following questions in your report:
  - What type of measure or measures have you considered to prevent overfitting?
  - How could one understand that a model being trained starts to overfit?
  - Could we get rid of the search over the number of iterations (epochs) hyperparameter by setting it to a relatively high value and doing some additional work? What may this additional work be? (Hint: You can think of this question together with the first one.)
  - Is there a "best" learning rate value that outperforms the other tested learning values in all hyperparameter configurations? (e.g., it may always produce the smallest loss value and highest accuracy score among all of the tested hyperparameter configurations).
  - Is there a "best" activation function that outperforms the other tested activation functions in all hyperparameter configurations? (e.g., it may always produce the smallest loss value and highest accuracy score among all of the tested hyperparameter configurations).
  - What are the advantages and disadvantages of using a small learning rate?
  - What are the advantages and disadvantages of using a big learning rate?
  - Is it a good idea to use stochastic gradient descent learning with a very large dataset? What kind of problem or problems do you think could emerge?
  - In the given source code, the instance features are divided by 255 (Please recall that in an image, pixel values range between 0 and 255). Why may such an operation be necessary? What would happen if we did not perform this operation? (Hint: These values are indirectly fed into the activation functions (e.g., sigmoid, tanh) of the neuron units. What happens to the gradient values when these functions are fed with large values?)

# Part 4

Neural network literature has matured significantly in the last decade, especially thanks to advancements in computational hardware and the proliferation of software packages. To broaden the topic and familiarize you with further neural-network-related concepts, this part requires you to answer the following research questions in your report (please cite resources you have utilized for your answers):

- Please derive the cross-entropy loss function for a multiclass classification (# of classes $> 2$) problem step by step and provide all necessary mathematical background and assumptions briefly.
- Please derive the squared-error loss function for a regression problem requiring predicting a single scalar value (single output) step by step, and provide all necessary mathematical background and assumptions briefly.
- In Part 3, the implementation shuffles the training data instances before training. What are the advantages of shuffling the training data for batch gradient descent, stochastic gradient descent, and mini-batch gradient descent? Please discuss each of them separately. What kind of problems may occur if this data set is not shuffled before training?
- In Part 3, before training, we preprocess the validation and test data sets with the statistics (mean, standard deviation, etc.) calculated from the training dataset. Why do you think we do it this way? Can't we just combine all datasets into a single dataset and then calculate the necessary metrics in order to normalize all the datasets? In other words, can't we just utilize the statistics calculated by taking all datasets into account for normalization?
- For classification, while assessing how a model is good at a given task or comparing multiple methods' performances, we consider a performance metric such as accuracy or precision. Although we are optimizing methods with respect to the loss functions (e.g., cross-entropy), we do not utilize their values for comparison or assessment. Why do we do so? What is the relationship between the loss values and performance metrics? Instead of the loss function, can't we just optimize our model with respect to a performance metric (e.g., accuracy)?
- In the literature, many weight-updating approaches with adaptive learning-rate scheduling have been proposed, such as RMSProp and Adam. Please explain these two approaches briefly and how they improve the naive gradient descent approach (with a fixed learning rate).
- What are the exploding and vanishing gradient problems in neural networks? In what circumstances could they occur?
- Approaches such as skip connections (residual connections), batch normalization, layer normalization, and Xavier initialization have been proposed to tackle the exploding and vanishing gradient problems. Please explain them and how they help circumvent the exploding and vanishing gradient problems briefly.
- To deal with the overfitting problem when using neural networks, one (regularization) approach is to utilize the weight decay approach that optimizes an augmented loss function:

$$L_{augmented} = L + \lambda * \sum_i ||w_i||^2$$

  where $L$ is the loss function for a given problem (e.g., mean cross-entropy), $w_i$ is the $i^{\text{th}}$ parameter of the network, and $\lambda$ is a scalar factor. This approach aims to keep network parameters close to 0 (it restricts the growth of parameters). Since weights are initialized close to 0, during learning, weights move away from 0; hence, they start contributing to the function defined by the network (the function gets more complex). If they remain close to 0, their contribution is very minimal, and the function defined remains simpler. Since weight decay keeps weights close to 0 by minimizing the right-hand side of the augmented loss function, and some weights have to grow due to the problem loss function, it can help the network implement the right function to solve the given ML problem (model complexity can be adjusted automatically). In the literature, there are several other approaches for the same purpose, such as skip connections (residual connections), dropout mechanism, and data augmentation. Please explain these approaches and how they help circumvent the overfitting problem with neural networks briefly.

- Typically, neural networks are trained with first-order optimization methods (methods that use the gradient information, e.g., gradient descent). :

$$W^{new} = W - \eta \nabla_W L$$

where $W$ is the vector of all weights in a neural network ($W = \begin{bmatrix} w_0 & w_1 & w_2 & ... \end{bmatrix}^T$), $\nabla_W L$ is the gradient vector of the $L$ with respect to $W$:

$$\nabla_W L = \begin{bmatrix} \frac{\partial L}{\partial w_0} & \frac{\partial L}{\partial w_1} & \frac{\partial L}{\partial w_3} & ... \end{bmatrix}^T$$

However, instead, second-order approaches could also be utilized (methods that utilize the second-order derivative information), such as Newton's method:

$$W^{new} = W - \eta H^{-1} \nabla_W L$$

where $H^{-1}$ is the inverse of the Hessian matrix ($H$) of $L$ with respect to $W$:

$$H = \begin{bmatrix} \frac{\partial^2 L}{\partial w_0 \partial w_0} & \frac{\partial^2 L}{\partial w_0 \partial w_1} & \frac{\partial^2 L}{\partial w_0 \partial w_2} & \frac{\partial^2 L}{\partial w_0 \partial w_3} & \cdots \\ \frac{\partial^2 L}{\partial w_1 \partial w_0} & \frac{\partial^2 L}{\partial w_1 \partial w_1} & \frac{\partial^2 L}{\partial w_1 \partial w_2} & \frac{\partial^2 L}{\partial w_1 \partial w_3} & \cdots \\ \frac{\partial^2 L}{\partial w_2 \partial w_0} & \frac{\partial^2 L}{\partial w_2 \partial w_1} & \frac{\partial^2 L}{\partial w_2 \partial w_2} & \frac{\partial^2 L}{\partial w_2 \partial w_3} & \cdots \\ \frac{\partial^2 L}{\partial w_3 \partial w_0} & \frac{\partial^2 L}{\partial w_3 \partial w_1} & \frac{\partial^2 L}{\partial w_3 \partial w_2} & \frac{\partial^2 L}{\partial w_3 \partial w_3} & \cdots \\ . & . & . & . & ... \\ . & . & . & . & ... \end{bmatrix}$$

Please discuss the advantages and disadvantages of the utilization of second-order approaches for neural network training briefly.

# Regulations

1. You are expected to write your code in Python using PyTorch, Numpy, copy, and Matplotlib libraries.

2. For Part 3, please include a README file explaining the source code file/s (along with their purpose) and how to run them if you have divided your work into multiple files.

3. The test set is only used to assess a model's unbiased generalization performance. You must not use it while training, performing hyperparameter tuning, or preprocessing the data instances of training and validation parts.

4. Falsifying results or changing the composition of training, validation, and test data is strictly forbidden, and you will receive 0 if this is the case. Your programs will be examined to see if you have actually reached the results and if it is working correctly.

5. **Late Submission:** You have a total of 72 late hours for all homework (HW1, HW2, HW3) without receiving a penalty. As soon as you have depleted your quota, penalization will be in effect. The late submission penalty will be calculated using $5d^2$, that is, 1 day (0-24 hours) late submission will cost you 5 points, 2 days (24-48 hours) will cost you 20 points, and 3 days (48-72 hours) will cost you 45 points. **No late submission is accepted after 3 days from the deadline of the assignment.**

6. **Cheating:** Using any piece of code that is not your own is strictly forbidden and constitutes cheating. This includes friends, previous homework, or the internet. However, example code snippets shared on PyTorch's website can be used. **We have a zero-tolerance policy for cheating**. People involved in cheating will be punished according to university regulations. If you have used LLM assisted code generation for your submission, please make sure that you have provided all the interaction details with it/them in the report (for which parts of the assignment you have used them, all the prompts utilized, further prompts for corrections/editing, all the code changes done by you if needed, for what percent of the whole assignment implementation have you got assistance from LLMs, .etc). We highly recommend you do your research and implementation without any LLM assistance, as their outputs may not be 100% correct, which may lead to wrong conclusions/results, and fact-checking is a must with them. Also, they can prevent you from getting hands-on experience in ML at both the knowledge level and implementation level, as they pose a shortcut for the learning process.

7. **Discussion:** You must follow ODTUClass for discussions and possible updates/corrections on a daily basis. If you think that your question concerns everyone, please ask them on ODTUClass.

8. **Evaluation:** Your assignment is going to be graded manually.

# When can I train my network?

You can always use your local computer's CPU/GPU to train the models. Another option could be Google Colab even though it imposes some restrictions on free users.

# Submission

Submission will be done via the ODTUClass system. For Part 1, you are expected to upload a single pdf file named **part1.pdf** which includes derivation rules, the completed `mlp_classification_backpropagation.py` and `mlp_regression_backpropagation.py` files. For Part 2 you are expected to upload the completed **part2_mlpclassification.py** and **part2_mlpregression.py** files. For Part 3 and Part 4, you are expected to upload your report (**report.pdf**) along with Part 3's implementation. For the implementations, if you have aggregated all your work into a single file, you can upload it under the name **part3.py** or if you have divided your implementation into multiple files, all of them should be compressed (along with the README file) into a single file (**part3.zip**) and this file should be submitted. Your are expected to prepare your reports using Latex.

# References


1. Kurt Hornik, Maxwell Stinchcombe, Halbert White, Multilayer feedforward networks are universal approximators, Neural Networks, Volume 2, Issue 5, 1989.

2. https://pytorch.org/docs/stable/notes/randomness.html

3. Vapnik: Vapnik, Vladimir (1999). The Nature of Statistical Learning Theory. Springer: New York.

4. Alpaydın: Introduction to Machine Learning, 4th edition, Ethem Alpaydın.

5. Bishop: Pattern Recognition and Machine Learning, Christopher M. Bishop.

6. For the `backpropagation` algorithm, you can refer to:

    - Lecture notes

- Bishop Chapter 5.
- Alpaydın Chapter 11.
- Neural Networks and Learning Machines, 3rd edition, Simon Haykin, Chapter 4.

7. Announcements Page
8. Discussion Forum